

Université Hassan 1er
ENSA Berrechid

Année Universitaire : 2025-2026

COMPTE RENDU

Bases de Données Avancées

SYNTHÈSE GLOBALE DE TOUS LES TP_s

Réalisée par :
Bouabidi Hind

Encadré par :
Pr. Hamid HRIMECH

Filière : ISIBD - S6

Table des matières

1	TP 1 : Vues et Index	4
1.1	Introduction	4
1.2	Exercice 1 : Les Vues	4
1.2.1	Création de la vue V_EMP	4
1.2.2	Sélection avec conditions	4
1.2.3	Utilisation de WITH CHECK OPTION	4
1.3	Exercice 2 : Les Index	4
1.4	Exercices Avancés : Sécurité et Optimisation	5
1.4.1	Exercice 4 : Vue en lecture seule (WITH READ ONLY)	5
1.4.2	Exercice 5 : Sécurité et Abstraction	5
1.4.3	Exercice 6 : Analyse globale (Contexte International)	5
2	TD 2 : Sous-requêtes	6
2.1	Introduction	6
2.2	Exercice 1 : Gestion des Étudiants	6
2.2.1	Question 1 : Étudiants sans date de naissance	6
2.2.2	Question 2 : Filtrage par motif (Nom avec 'a')	6
2.2.3	Question 3 : Matières par coefficient	6
2.2.4	Question 4 : Notes identiques (Étudiant 1 vs 2)	6
2.2.5	Question 5 : Notes supérieures (Étudiant 1 vs 2)	7
2.2.6	Question 6 : Notes inférieures (Étudiant 1 vs 9)	7
2.2.7	Question 7 : Étudiants sans aucune note	7
2.2.8	Question 8 : Âge moyen par sexe au 01/01/2000	7
2.2.9	Question 9 : Enseignants d'Histoire	7
2.2.10	Question 10 : Étudiants sans note en Sociologie	7
2.3	Exercice 2 : Schéma EMP / DEPT	7
2.3.1	Question 11 : Lieux des départements avec commission	7
2.3.2	Question 12 : Départements avec au moins un ingénieur	8
2.3.3	Question 13 : Départements sans ingénieur	8
2.3.4	Question 14 : Salaire inférieur à 50% du supérieur (Synchronisée)	8
2.3.5	Question 15 : Salaire supérieur à tous les commerciaux	8
2.3.6	Question 16 : Salaire supérieur aux commerciaux du même département	8
2.3.7	Question 17 : Employés ayant le salaire maximum de leur poste	8
3	TP : Fondamentaux du PL/SQL et Types Complexes	9
3.1	Introduction	9
3.2	Déclaration et Portée des Variables	9
3.2.1	Exercice 1 : Variables Scalaires	9
3.2.2	Exercice 2 : Portée des Variables (Blocs imbriqués)	9
3.3	Attributs de Type Dynamique	10

3.3.1	Exercice 3 : Utilisation de %TYPE	10
3.3.2	Exercice 4 : Utilisation de %ROWTYPE	10
3.4	Structures de Données Avancées	10
3.4.1	Exercice 5 : RECORD Personnalisé	10
3.4.2	Exercice 6 : Table Associative (Collections)	10
3.5	Exercice Final de Synthèse	11
4	Manipulation des Données et Structures de Contrôle	12
4.1	Introduction	12
4.2	Exercices de Programmation PL/SQL	12
4.2.1	Exercice 1 : Insertion automatique (Boucle FOR)	12
4.2.2	Exercice 2 : Calcul de somme cumulative	12
4.2.3	Exercice 3 : Filtrage avec condition IF	13
4.2.4	Exercice 4 : Calcul du Factoriel	13
4.2.5	Exercice 5 : Gestion de la table Etudiant	13
4.2.6	Exercice 6 : Boucle WHILE	13
4.3	Conclusion	14
5	TP 5 : Manipulation des Curseurs	15
5.1	Introduction	15
5.2	Exercices : Curseurs Implicites et Attributs	15
5.2.1	Exercice 1 : Utilisation de SQL%ROWCOUNT	15
5.3	Exercices : Curseurs Explicites	15
5.3.1	Exercice 2 : Liste des commandes (Base Usine)	15
5.3.2	Exercice 3 : Commandes supérieures à la moyenne	16
5.4	Exercices : Gestion Semi-Automatique (Boucle FOR)	16
5.4.1	Exercice 4 : Liste des employés à Créteil	16
5.4.2	Exercice 5 : Subordonnés de "Guimezanes"	16
6	TP : Logique Conditionnelle et Curseurs Avancés	18
6.1	Introduction	18
6.2	Exercices : Structures Conditionnelles	18
6.2.1	Exercice 1 : Permutation de deux nombres	18
6.2.2	Exercice 2 : Vérification de parité	18
6.2.3	Exercice 3 : Détection du Week-end	19
6.3	Exercices : Curseurs et Manipulation de Tables	19
6.3.1	Exercice 4 : Copie de table (Clients vers tmp_clients)	19
6.3.2	Exercice 5 : Suivi des mises à jour (SQL%NOTFOUND)	19
6.3.3	Exercice 6 : Augmentation sélective (WHERE CURRENT OF)	19
6.4	Conclusion	20
7	TP 6 : Triggers et Contraintes d'Intégrité Complexes	21
7.1	Introduction	21
7.2	Mise en place de l'environnement	21
7.3	Implémentation des 9 Règles	21
7.3.1	Règle 1 : Interdiction de modifier note_min	21
7.3.2	Règle 2 : Contrôle de l'effectif maximum	21
7.3.3	Règle 3 : Création d'examen uniquement si inscriptions existent	22
7.3.4	Règle 4 : Date d'examen postérieure à l'inscription	22
7.3.5	Règle 5 : Détection de circuit dans les prérequis	22
7.3.6	Règle 6 : Vérification des prérequis lors de l'inscription	23

7.3.7	Règle 7 : Mise à jour automatique de la moyenne	23
7.3.8	Règle 8 : Modification de note_min interdite si elle invalide des inscriptions	24
7.3.9	Règle 9 : Modification de effecMax sans invalider d'inscriptions	24
7.4	Gestion des exceptions personnalisées	24
7.5	Conclusion	25

1

TP 1 : Vues et Index

1.1 Introduction

Ce TP traite de la manipulation des vues pour simplifier les requêtes complexes et de l'utilisation des index pour optimiser les performances d'accès aux données.

1.2 Exercice 1 : Les Vues

1.2.1 Création de la vue V_EMP

La vue V_EMP regroupe les informations de l'employé et de son département avec le calcul des gains totaux.

```
1 CREATE OR REPLACE VIEW V_EMP AS
2 SELECT Matr, NomE, NumDept, (NVL(Commission, 0) + Salaire) AS GAINS, Lieu
3 FROM EMP e, DEPT d
4 WHERE e.NumDept = d.NumDept;
```

1.2.2 Sélection avec conditions

Extraction des employés dont les gains dépassent 10 000.

```
1 SELECT * FROM V_EMP WHERE GAINS > 10000;
```

1.2.3 Utilisation de WITH CHECK OPTION

Mise en place d'une contrainte sur la vue pour empêcher l'insertion de données ne respectant pas le filtre du département 10.

```
1 CREATE OR REPLACE VIEW VEMP10 AS
2 SELECT * FROM EMP WHERE NumDept = 10
3 WITH CHECK OPTION;
```

1.3 Exercice 2 : Les Index

Optimisation des recherches sur le nom et le poste.

```
1 CREATE INDEX idx_emp_nom ON EMP(NomE);
2 CREATE INDEX idx_emp_dept_post ON EMP(NumDept, Poste);
```

1.4 Exercices Avancés : Sécurité et Optimisation

1.4.1 Exercice 4 : Vue en lecture seule (WITH READ ONLY)

L'option WITH READ ONLY interdit toute opération de mise à jour (DML) à travers la vue, garantissant l'intégrité des données sources.

```
1 CREATE OR REPLACE VIEW emp_readonly AS
2 SELECT * FROM Employe
3 WITH READ ONLY;
4
5 -- Test de suppression (Genere une erreur)
6 DELETE FROM emp_readonly WHERE emp_id = 1;
```

1.4.2 Exercice 5 : Sécurité et Abstraction

Création d'une vue masquant les données sensibles (comme le salaire) pour un usage public.

```
1 CREATE OR REPLACE VIEW empxpublic AS
2 SELECT emp_id, nom, dept_id
3 FROM Employe;
```

Discussion : Une vue n'améliore pas directement les performances de calcul (elle exécute la requête sous-jacente), mais elle simplifie l'accès et renforce la sécurité.

1.4.3 Exercice 6 : Analyse globale (Contexte International)

1. Architecture de sécurité proposée :

- **Vues par département** : Créer des vues filtrées par `dept_id` pour les managers (ex : `WHERE dept_id = SYSDATE_DEPT`).
- **Masquage de colonnes** : Utiliser des vues sans la colonne `salaire` pour les employés standards.

2. Limites des vues classiques : Sur des centaines de milliers de lignes, une vue simple peut être lente car elle recalcule la jointure à chaque appel.

3. Intérêt des vues matérialisées : Contrairement aux vues classiques, les vues matérialisées stockent physiquement le résultat. Elles sont idéales pour les grands volumes car elles évitent les calculs répétitifs, au prix d'une mise à jour (refresh) périodique.

4. Vues vs Privilèges (GRANT) : Les privilèges contrôlent l'accès à la table entière, alors que les vues permettent un contrôle plus fin au niveau des lignes (filtrage horizontal) et des colonnes (filtrage vertical).

5. Stratégie d'optimisation : Pour un système optimal, il est recommandé de combiner :

- Des **Index** sur les colonnes de jointure (`dept_id`).
- Le **Partitionnement** des tables par site géographique.
- Des **Vues Matérialisées** pour les rapports de synthèse RH.

2

TD 2 : Sous-requêtes

2.1 Introduction

Ce TD explore l'utilisation des sous-interrogations SQL pour résoudre des problèmes complexes. Nous abordons les sous-requêtes simples (retournant une valeur unique), les sous-requêtes multiples (utilisant IN, ANY, ALL) et les sous-requêtes synchronisées.

2.2 Exercice 1 : Gestion des Étudiants

2.2.1 Question 1 : Étudiants sans date de naissance

Affichage du numéro, nom et sexe des étudiants dont la date de naissance est inconnue.

```
1 SELECT Numetu, Nometu, Cdsexe
2 FROM ETUDIANT
3 WHERE Dtnaiss IS NULL;
```

2.2.2 Question 2 : Filtrage par motif (Nom avec 'a')

```
1 SELECT Nometu, Dtnaiss
2 FROM ETUDIANT
3 WHERE Nometu LIKE '_a%';
```

2.2.3 Question 3 : Matières par coefficient

```
1 SELECT Nomat FROM MATIERE
2 WHERE Coeff BETWEEN 1 AND 2;
```

2.2.4 Question 4 : Notes identiques (Étudiant 1 vs 2)

```
1 SELECT Note FROM NOTES
2 WHERE Numetu = 1
3 AND Note IN (SELECT Note FROM NOTES WHERE Numetu = 2);
```

2.2.5 Question 5 : Notes supérieures (Étudiant 1 vs 2)

```
1 SELECT Note FROM NOTES
2 WHERE Numetu = 1
3 AND Note > ANY (SELECT Note FROM NOTES WHERE Numetu = 2);
```

2.2.6 Question 6 : Notes inférieures (Étudiant 1 vs 9)

```
1 SELECT Note FROM NOTES
2 WHERE Numetu = 1
3 AND Note < ALL (SELECT Note FROM NOTES WHERE Numetu = 9);
```

2.2.7 Question 7 : Étudiants sans aucune note

```
1 SELECT * FROM ETUDIANT
2 WHERE Numetu NOT IN (SELECT DISTINCT Numetu FROM NOTES);
```

2.2.8 Question 8 : Âge moyen par sexe au 01/01/2000

```
1 SELECT Cdsexe, AVG(MONTHS_BETWEEN(TO_DATE('01/01/2000', 'DD/MM/YYYY'),
2 Dtnaiss)/12) AS Age_Moyen
3 FROM ETUDIANT
4 GROUP BY Cdsexe;
```

2.2.9 Question 9 : Enseignants d'Histoire

```
1 SELECT Nomens, Grade FROM ENSEIGNANT
2 WHERE Numens IN (SELECT Numens FROM MATIERE WHERE Nomat = 'Histoire');
```

2.2.10 Question 10 : Étudiants sans note en Sociologie

```
1 SELECT Nometu, Numetu FROM ETUDIANT
2 WHERE Numetu NOT IN (
3     SELECT Numetu FROM NOTES
4     WHERE Numat IN (SELECT Numat FROM MATIERE WHERE Nomat = 'Sociologie')
5 );
```

2.3 Exercice 2 : Schéma EMP / DEPT

2.3.1 Question 11 : Lieux des départements avec commission

```
1 SELECT Lieu FROM DEPT
2 WHERE NumDept IN (SELECT DISTINCT NumDept FROM EMP WHERE Commission IS NOT
3 NULL);
```


2.3.2 Question 12 : Départements avec au moins un ingénieur

```
1 SELECT NomDept, Lieu FROM DEPT
2 WHERE NumDept IN (SELECT NumDept FROM EMP WHERE Poste = 'INGENIEUR');
```

2.3.3 Question 13 : Départements sans ingénieur

```
1 SELECT NomDept, Lieu FROM DEPT
2 WHERE NumDept NOT IN (SELECT NumDept FROM EMP WHERE Poste = 'INGENIEUR');
```

2.3.4 Question 14 : Salaire inférieur à 50% du supérieur (Synchronisée)

```
1 SELECT NomE FROM EMP e
2 WHERE Salaire < (SELECT Salaire * 0.5 FROM EMP WHERE Matr = e.Sup);
```

2.3.5 Question 15 : Salaire supérieur à tous les commerciaux

```
1 SELECT NomE FROM EMP
2 WHERE Salaire > ALL (SELECT Salaire FROM EMP WHERE Poste = 'COMMERCIAL');
```

2.3.6 Question 16 : Salaire supérieur aux commerciaux du même département

```
1 -- Version incluant les departements sans commerciaux
2 SELECT NomE FROM EMP e1
3 WHERE Salaire > ALL (
4     SELECT Salaire FROM EMP e2
5     WHERE e2.NumDept = e1.NumDept AND e2.Poste = 'COMMERCIAL'
6 );
```

2.3.7 Question 17 : Employés ayant le salaire maximum de leur poste

```
1 SELECT NomE, Salaire, Poste FROM EMP e1
2 WHERE Salaire = (SELECT MAX(Salaire) FROM EMP e2 WHERE e2.Poste = e1.Poste);
```

3

TP : Fondamentaux du PL/SQL et Types Complexes

3.1 Introduction

Ce TP explore les bases de la programmation procédurale avec Oracle, en mettant l'accent sur la déclaration rigoureuse des variables, l'utilisation des attributs dynamiques et la manipulation de structures de données avancées comme les RECORD et les collections.

3.2 Déclaration et Portée des Variables

3.2.1 Exercice 1 : Variables Scalaires

Déclaration de différents types de données (VARCHAR2, NUMBER, DATE, BOOLEAN).

```
1 DECLARE
2     v_nom VARCHAR2(20) := 'Hind';
3     v_age NUMBER(2) := 21;
4     v_date DATE := SYSDATE;
5     v_bool BOOLEAN := TRUE;
6 BEGIN
7     DBMS_OUTPUT.PUT_LINE('Nom : ' || v_nom);
8     DBMS_OUTPUT.PUT_LINE('Date : ' || v_date);
9 END;
10 /
```

3.2.2 Exercice 2 : Portée des Variables (Blocs imbriqués)

Démonstration de la visibilité des variables dans les blocs parents et enfants.

```
1 DECLARE
2     v_global VARCHAR2(20) := 'GLOBAL';
3 BEGIN
4     DECLARE
5         v_local VARCHAR2(20) := 'LOCAL';
6         BEGIN
7             DBMS_OUTPUT.PUT_LINE(v_global); -- Accessible
8             DBMS_OUTPUT.PUT_LINE(v_local);  -- Accessible
9         END;
10    -- DBMS_OUTPUT.PUT_LINE(v_local); -- Erreur : v_local est invisible ici
11 END;
12 /
```

3.3 Attributs de Type Dynamique

3.3.1 Exercice 3 : Utilisation de %TYPE

Sécurisation du typage en se basant sur la structure de la table `employees`.

```
1 DECLARE
2     v_emp_nom employees.nom%TYPE;
3     v_salaire employees.salaire%TYPE;
4 BEGIN
5     SELECT nom, salaire INTO v_emp_nom, v_salaire
6     FROM employees WHERE id = 1;
7     DBMS_OUTPUT.PUT_LINE(v_emp_nom || ' gagne ' || v_salaire);
8 END;
9 /
```

3.3.2 Exercice 4 : Utilisation de %ROWTYPE

Déclaration d'une structure complète correspondant à une ligne de la table.

```
1 DECLARE
2     v_emp_rec employees%ROWTYPE;
3 BEGIN
4     SELECT * INTO v_emp_rec FROM employees WHERE id = 1;
5     DBMS_OUTPUT.PUT_LINE('ID: ' || v_emp_rec.id || ' Nom: ' || v_emp_rec.nom
6     );
7 END;
8 /
```

3.4 Structures de Données Avancées

3.4.1 Exercice 5 : RECORD Personnalisé

Création d'un type structuré regroupant des informations de plusieurs sources.

```
1 DECLARE
2     TYPE t_emp_info IS RECORD (
3         nom VARCHAR2(50),
4         poste VARCHAR2(50),
5         salaire NUMBER
6     );
7     v_info t_emp_info;
8 BEGIN
9     v_info.nom := 'Bouabidi';
10    v_info.poste := 'Ingenieur';
11    v_info.salaire := 15000;
12    DBMS_OUTPUT.PUT_LINE('Poste : ' || v_info.poste);
13 END;
14 /
```

3.4.2 Exercice 6 : Table Associative (Collections)

Manipulation d'une collection indexée par des entiers.

```

1 DECLARE
2     TYPE t_tab_noms IS TABLE OF VARCHAR2(50) INDEX BY BINARY_INTEGER;
3     v_noms t_tab_noms;
4 BEGIN
5     v_noms(1) := 'Salma';
6     v_noms(2) := 'Kawatar';
7     v_noms(3) := 'Maryam';
8
9     FOR i IN 1..3 LOOP
10        DBMS_OUTPUT.PUT_LINE('Nom ' || i || ' : ' || v_noms(i));
11    END LOOP;
12 END;
13 /

```

3.5 Exercice Final de Synthèse

Combinaison d'un RECORD personnalisé et d'une TABLE de ce record pour calculer des statistiques.

```

1 DECLARE
2     TYPE t_synth_rec IS RECORD (id NUMBER, nom VARCHAR2(50), sal NUMBER);
3     TYPE t_synth_coll IS TABLE OF t_synth_rec INDEX BY BINARY_INTEGER;
4     v_liste t_synth_coll;
5     v_total NUMBER := 0;
6     v_max NUMBER := 0;
7 BEGIN
8     -- Insertion de 5 employes fictifs
9     FOR i IN 1..5 LOOP
10        v_liste(i).id := i;
11        v_liste(i).nom := 'Emp_' || i;
12        v_liste(i).sal := 4000 + (i * 200);
13        v_total := v_total + v_liste(i).sal;
14        IF v_liste(i).sal > v_max THEN v_max := v_liste(i).sal; END IF;
15    END LOOP;
16
17    DBMS_OUTPUT.PUT_LINE('Salaire Moyen : ' || (v_total / 5));
18    DBMS_OUTPUT.PUT_LINE('Salaire Maximum : ' || v_max);
19 END;
20 /

```

=====

4

Manipulation des Données et Structures de Contrôle

4.1 Introduction

L'objectif de ce TP est de manipuler les structures de contrôle (boucles, conditions) du langage PL/SQL pour automatiser des tâches répétitives et interagir efficacement avec les tables de la base de données Oracle.

4.2 Exercices de Programmation PL/SQL

4.2.1 Exercice 1 : Insertion automatique (Boucle FOR)

Ce bloc permet d'insérer dynamiquement 100 lignes dans une table Nombre.

Listing 4.1 – Insertion de 1 à 100

```
1 BEGIN
2   FOR i IN 1..100 LOOP
3     INSERT INTO Nombre (n) VALUES (i);
4   END LOOP;
5   COMMIT;
6 END;
7 /
```

4.2.2 Exercice 2 : Calcul de somme cumulative

Calcul de la somme des entiers de 1 à 1000.

```
1 DECLARE
2   v_somme NUMBER := 0;
3 BEGIN
4   FOR i IN 1..1000 LOOP
5     v_somme := v_somme + i;
6   END LOOP;
7   DBMS_OUTPUT.PUT_LINE('La somme totale est : ' || v_somme);
8 END;
9 /
```

4.2.3 Exercice 3 : Filtrage avec condition IF

Affichage des nombres pairs entre 1 et 50.

```
1 BEGIN
2   FOR i IN 1..50 LOOP
3     IF MOD(i, 2) = 0 THEN
4       DBMS_OUTPUT.PUT_LINE('Nombre pair : ' || i);
5     END IF;
6   END LOOP;
7 END;
8 /
```

4.2.4 Exercice 4 : Calcul du Factoriel

```
1 DECLARE
2   v_n NUMBER := 5;
3   v_fact NUMBER := 1;
4 BEGIN
5   FOR i IN 1..v_n LOOP
6     v_fact := v_fact * i;
7   END LOOP;
8   DBMS_OUTPUT.PUT_LINE('Le factoriel de ' || v_n || ' est : ' || v_fact);
9 END;
10 /
```

4.2.5 Exercice 5 : Gestion de la table Etudiant

```
1 BEGIN
2   FOR i IN 1..10 LOOP
3     INSERT INTO Etudiant (id, nom, note)
4     VALUES (i, 'Etudiant_' || i, 10 + i);
5   END LOOP;
6   COMMIT;
7 END;
8 /
```

4.2.6 Exercice 6 : Boucle WHILE

Décompte de 10 à 1.

```
1 DECLARE
2   v_cpt NUMBER := 10;
3 BEGIN
4   WHILE v_cpt >= 1 LOOP
5     DBMS_OUTPUT.PUT_LINE('Compteur : ' || v_cpt);
6     v_cpt := v_cpt - 1;
7   END LOOP;
8 END;
9 /
```

4.3 Conclusion

L'utilisation des blocs PL/SQL permet d'intégrer une logique métier complexe directement au sein du SGBD. Nous avons vu que les boucles facilitent grandement les opérations de masse sur les données.=====

5

TP 5 : Manipulation des Curseurs

5.1 Introduction

Un curseur est une zone mémoire (contexte SQL) permettant de traiter individuellement chaque ligne renvoyée par une requête **SELECT**[cite : 185]. Nous distinguons deux types :

- **Curseur implicite** : Géré automatiquement par Oracle pour les instructions SQL simples[cite : 15].
- **Curseur explicite** : Déclaré par le développeur pour un contrôle fin du parcours des données (OPEN, FETCH, CLOSE)[cite : 188, 191].

5.2 Exercices : Curseurs Implicites et Attributs

5.2.1 Exercice 1 : Utilisation de SQL%ROWCOUNT

Ce bloc permet de compter le nombre de lignes affectées par une mise à jour via un curseur implicite[cite : 15].

```
1 BEGIN
2     UPDATE employees SET salaire = salaire + 100 WHERE departement_id = 1;
3     IF SQL%FOUND THEN
4         DBMS_OUTPUT.PUT_LINE('Nombre de lignes mises a jour : ' || SQL%
5                               ROWCOUNT);
6     ELSE
7         DBMS_OUTPUT.PUT_LINE('Aucune ligne n''a ete modifiee');
8     END IF;
9 END;
```

5.3 Exercices : Curseurs Explicites

5.3.1 Exercice 2 : Liste des commandes (Base Usine)

Affichage de toutes les commandes en utilisant un curseur explicite[cite : 178].

```
1 DECLARE
2     CURSOR c_commandes IS SELECT Numcmd, Mncmd FROM Commande;
3     v_num Commande.Numcmd%TYPE;
4     v_montant Commande.Mncmd%TYPE;
5 BEGIN
6     OPEN c_commandes;
```



```

7      LOOP
8          FETCH c_commandes INTO v_num, v_montant;
9          EXIT WHEN c_commandes%NOTFOUND;
10         DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_num || ' - Montant : ' ||
                                v_montant);
11     END LOOP;
12     CLOSE c_commandes;
13 END;
14 /

```

5.3.2 Exercice 3 : Commandes supérieures à la moyenne

Utilisation d'un curseur pour filtrer les données par rapport à un calcul agrégé[cite : 178].

```

1 DECLARE
2     CURSOR c_sup_moy IS
3         SELECT Numcmd, Mncmd FROM Commande
4         WHERE Mncmd > (SELECT AVG(Mncmd) FROM Commande);
5 BEGIN
6     FOR rec IN c_sup_moy LOOP
7         DBMS_OUTPUT.PUT_LINE('Commande au-dessus de la moyenne : ' || rec.
                                Numcmd);
8     END LOOP;
9 END;
10 /

```

5.4 Exercices : Gestion Semi-Automatique (Boucle FOR)

5.4.1 Exercice 4 : Liste des employés à Créteil

Parcours des employés avec une jointure dans le curseur pour filtrer par lieu[cite : 182].

```

1 DECLARE
2     CURSOR c_creteil IS
3         SELECT NomE FROM Emp e
4         JOIN Dept d ON e.NumD = d.NumD
5         WHERE d.Lieu = 'Creteil';
6 BEGIN
7     FOR emp_rec IN c_creteil LOOP
8         DBMS_OUTPUT.PUT_LINE('Employe travaillant a Creteil : ' || emp_rec.
                                NomE);
9     END LOOP;
10 END;
11 /

```

5.4.2 Exercice 5 : Subordonnés de "Guimezanes"

Utilisation d'un curseur pour identifier les relations hiérarchiques[cite : 182].

```

1 DECLARE
2     CURSOR c_subordonnes IS
3         SELECT NomE FROM Emp
4         WHERE NumS = (SELECT NumE FROM Emp WHERE NomE = 'Guimezanes');
5 BEGIN
6     DBMS_OUTPUT.PUT_LINE('Subordonnes de Guimezanes :');
7     FOR rec IN c_subordonnes LOOP

```

```
8         DBMS_OUTPUT.PUT_LINE(' - ' || rec.NomE);
9     END LOOP;
10 END;
11 /
```

6

TP : Logique Conditionnelle et Curseurs Avancés

6.1 Introduction

Ce TP aborde dans une première phase la logique algorithmique en PL/SQL (structures conditionnelles) avant de traiter des cas complexes de manipulation de données via des curseurs, notamment la copie de tables et la mise à jour avec clause de positionnement.

6.2 Exercices : Structures Conditionnelles

6.2.1 Exercice 1 : Permutation de deux nombres

Objectif : Réorganiser deux variables pour stocker la plus petite dans `small_nbr`.

```
1 DECLARE
2     v_n1 NUMBER := 25;
3     v_n2 NUMBER := 5;
4     v_temp NUMBER;
5     v_small NUMBER;
6     v_big NUMBER;
7 BEGIN
8     IF v_n1 < v_n2 THEN
9         v_small := v_n1; v_big := v_n2;
10    ELSE
11        v_small := v_n2; v_big := v_n1;
12    END IF;
13    DBMS_OUTPUT.PUT_LINE('Small: ' || v_small || ' | Big: ' || v_big);
14 END;
15 /
```

6.2.2 Exercice 2 : Vérification de parité

```
1 DECLARE
2     v_nbr NUMBER := 3;
3 BEGIN
4     IF MOD(v_nbr, 2) = 0 THEN
5         DBMS_OUTPUT.PUT_LINE('Le nombre ' || v_nbr || ' est pair.');
```

```

9  END;
10 /

```

6.2.3 Exercice 3 : Détection du Week-end

```

1  DECLARE
2      v_date DATE := TO_DATE('02-03-2024', 'DD-MM-YYYY');
3      v_jour VARCHAR2(20);
4  BEGIN
5      v_jour := TRIM(TO_CHAR(v_date, 'DAY', 'NLS_DATE_LANGUAGE = ENGLISH'));
6      IF v_jour IN ('SATURDAY', 'SUNDAY') THEN
7          DBMS_OUTPUT.PUT_LINE('La date tombe le week-end (' || v_jour || ').');
8      ELSE
9          DBMS_OUTPUT.PUT_LINE('Jour de semaine : ' || v_jour);
10     END IF;
11 END;
12 /

```

6.3 Exercices : Curseurs et Manipulation de Tables

6.3.1 Exercice 4 : Copie de table (Clients vers tmp_clients)

Utilisation d'un curseur pour transférer les enregistrements.

```

1  DECLARE
2      CURSOR c_clients IS SELECT * FROM clients;
3  BEGIN
4      FOR r IN c_clients LOOP
5          INSERT INTO tmp_clients (client_id, nom, ville, age)
6              VALUES (r.client_id, r.nom, r.ville, r.age);
7      END LOOP;
8      COMMIT;
9  END;
10 /

```

6.3.2 Exercice 5 : Suivi des mises à jour (SQL%NOTFOUND)

Test de l'impact d'une instruction UPDATE sur la table pays.

```

1  BEGIN
2      UPDATE pays SET nom = UPPER(nom) WHERE pays_id = 1001;
3      IF SQL%NOTFOUND THEN
4          DBMS_OUTPUT.PUT_LINE('Aucun pays avec cet ID.');
```

6.3.3 Exercice 6 : Augmentation sélective (WHERE CURRENT OF)

Mise à jour du salaire des employés du département 8 avec un curseur de mise à jour.

```

1 DECLARE
2     CURSOR c_emp_dept8 IS
3         SELECT salaire FROM employees
4         WHERE departement_id = 8
5         FOR UPDATE;
6 BEGIN
7     FOR r IN c_emp_dept8 LOOP
8         IF r.salaire < 5000 THEN
9             UPDATE employees SET salaire = salaire + 100
10            WHERE CURRENT OF c_emp_dept8;
11        ELSE
12            UPDATE employees SET salaire = salaire + 50
13            WHERE CURRENT OF c_emp_dept8;
14        END IF;
15    END LOOP;
16    COMMIT;
17 END;
18 /

```

6.4 Conclusion

Ce TP démontre que le PL/SQL permet non seulement de traiter des flux de données complexes via les curseurs, mais aussi d'intégrer des règles de gestion précises (comme le calcul de dates ou la parité) au sein même du moteur de base de données.

7

TP 6 : Triggers et Contraintes d'Intégrité Complexes

7.1 Introduction

Les triggers sont des blocs PL/SQL qui se déclenchent automatiquement lors d'événements spécifiques (INSERT, UPDATE, DELETE). Ils sont essentiels pour garantir la cohérence des données lorsque les contraintes classiques (Check, FK) sont insuffisantes.

7.2 Mise en place de l'environnement

Le schéma d'étude concerne la gestion des inscriptions universitaires avec les tables : `etudiant`, `module`, `prerequis`, `inscription`, `examen` et `passer`.

7.3 Implémentation des 9 Règles

7.3.1 Règle 1 : Interdiction de modifier `note_min`

```
1 CREATE OR REPLACE TRIGGER trg_prerequis_no_update_note_min
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 BEGIN
5     RAISE_APPLICATION_ERROR(-20001, 'Modification de note_min dans prerequis
6     interdite.');
```

7.3.2 Règle 2 : Contrôle de l'effectif maximum

```
1 CREATE OR REPLACE TRIGGER trg_inscription_check_effectif
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     v_inscrits NUMBER;
6     v_effecMax NUMBER;
7 BEGIN
8     SELECT effecMax INTO v_effecMax FROM module WHERE code_module = :NEW.
9     code_module;
```

```

9      SELECT COUNT(*) INTO v_inscrits FROM inscription WHERE code_module = :
      NEW.code_module;
10
11      IF v_inscrits >= v_effecMax THEN
12          RAISE_APPLICATION_ERROR(-20002, 'Effectif maximum atteint pour le
      module ' || :NEW.code_module);
13      END IF;
14  END;
15  /

```

7.3.3 Règle 3 : Création d'examen uniquement si inscriptions existent

```

1  CREATE OR REPLACE TRIGGER trg_examen_check_inscrits
2  BEFORE INSERT ON examen
3  FOR EACH ROW
4  DECLARE
5      v_nb NUMBER;
6  BEGIN
7      SELECT COUNT(*) INTO v_nb FROM inscription WHERE code_module = :NEW.
      code_module;
8      IF v_nb = 0 THEN
9          RAISE_APPLICATION_ERROR(-20003, 'Impossible de cr er un examen :
      aucun inscrit. ');
10     END IF;
11 END;
12 /

```

7.3.4 Règle 4 : Date d'examen postérieure à l'inscription

```

1  CREATE OR REPLACE TRIGGER trg_passer_check_date_inscription
2  BEFORE INSERT ON passer
3  FOR EACH ROW
4  DECLARE
5      v_date_insc DATE;
6      v_date_exam DATE;
7  BEGIN
8      SELECT date_inscription INTO v_date_insc
9      FROM inscription
10     WHERE id_etudiant = :NEW.id_etudiant
11     AND code_module = (SELECT code_module FROM examen WHERE id_examen = :NEW
      .id_examen);
12
13     SELECT date_examen INTO v_date_exam FROM examen WHERE id_examen = :NEW.
      id_examen;
14
15     IF v_date_insc >= v_date_exam THEN
16         RAISE_APPLICATION_ERROR(-20004, 'Inscription post rieuse ou gale
      la date d''examen. ');
17     END IF;
18 END;
19 /

```

7.3.5 Règle 5 : Détection de circuit dans les prérequis

```

1 CREATE OR REPLACE TRIGGER trg_prerequis_no_circuit
2 BEFORE INSERT OR UPDATE ON prerequis
3 FOR EACH ROW
4 DECLARE
5     v_circuit BOOLEAN;
6 BEGIN
7     v_circuit := existe_circuit(:NEW.prerequis, :NEW.module);
8     IF v_circuit THEN
9         RAISE_APPLICATION_ERROR(-20005, 'Circuit d tect : ' || :NEW.
            module || ' -> ' || :NEW.prerequis);
10    END IF;
11 END;
12 /

```

7.3.6 Règle 6 : Vérification des prérequis lors de l'inscription

```

1 CREATE OR REPLACE TRIGGER trg_inscription_check_prerequis
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     CURSOR c_prereq IS
6         SELECT prerequis, note_min FROM prerequis WHERE module = :NEW.
            code_module;
7     v_note NUMBER;
8 BEGIN
9     FOR p IN c_prereq LOOP
10        SELECT MAX(note) INTO v_note
11        FROM passer pa
12        JOIN examen e ON pa.id_examen = e.id_examen
13        WHERE pa.id_etudiant = :NEW.id_etudiant
14        AND e.code_module = p.prerequis;
15
16        IF v_note IS NULL OR v_note < p.note_min THEN
17            RAISE_APPLICATION_ERROR(-20006, 'Pr requis ' || p.prerequis ||
                ' non satisfait. ');
18        END IF;
19    END LOOP;
20 END;
21 /

```

7.3.7 Règle 7 : Mise à jour automatique de la moyenne

```

1 CREATE OR REPLACE TRIGGER trg_passer_update_moyenne
2 AFTER INSERT OR UPDATE OR DELETE ON passer
3 FOR EACH ROW
4 DECLARE
5     v_id_etudiant NUMBER;
6     v_moyenne NUMBER;
7 BEGIN
8     v_id_etudiant := NVL(:NEW.id_etudiant, :OLD.id_etudiant);
9
10    SELECT AVG(note) INTO v_moyenne
11    FROM passer
12    WHERE id_etudiant = v_id_etudiant;
13
14    UPDATE etudiant

```



```

15     SET moyenne = v_moyenne
16     WHERE id_etudiant = v_id_etudiant;
17 END;
18 /

```

7.3.8 Règle 8 : Modification de note_min interdite si elle invalide des inscriptions

```

1 CREATE OR REPLACE TRIGGER trg_prerequis_check_note_min_update
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 DECLARE
5     CURSOR c_etudiants IS
6         SELECT DISTINCT i.id_etudiant FROM inscription i WHERE i.code_module
7             = :NEW.module;
8     v_note_max NUMBER;
9 BEGIN
10    FOR etu IN c_etudiants LOOP
11        SELECT MAX(pa.note) INTO v_note_max
12        FROM passer pa
13        JOIN examen e ON pa.id_examen = e.id_examen
14        WHERE pa.id_etudiant = etu.id_etudiant
15        AND e.code_module = :NEW.prerequis;
16
17        IF v_note_max IS NULL OR v_note_max < :NEW.note_min THEN
18            RAISE_APPLICATION_ERROR(-20008, 'La nouvelle note_min
19                invaliderait des inscriptions existantes.');

```

7.3.9 Règle 9 : Modification de effecMax sans invalider d'inscriptions

```

1 CREATE OR REPLACE TRIGGER trg_module_check_effecMax_update
2 BEFORE UPDATE OF effecMax ON module
3 FOR EACH ROW
4 DECLARE
5     v_inscrits NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_inscrits
8     FROM inscription
9     WHERE code_module = :NEW.code_module;
10
11     IF :NEW.effecMax < v_inscrits THEN
12         RAISE_APPLICATION_ERROR(-20009, 'Le nouvel effectif max est
13             inf rieur au nombre d''inscrits actuels.');

```

7.4 Gestion des exceptions personnalisées

Les erreurs sont gérées via RAISE_APPLICATION_ERROR avec des codes entre -20000 et -20999.

7.5 Conclusion

Les triggers offrent une solution puissante pour implémenter des règles métier strictes. Cependant, leur utilisation doit être prudente pour éviter les problèmes de performance et de maintenance.